

C8 食谱 RAG 系统 AI 开发岗位面试文档

项目名称：尝尝咸淡 RAG 系统

代码目录：`code/C8`

文档用途：用于 AI 开发、LLM 应用开发、RAG 工程岗位面试准备

核心定位：基于本地食谱 Markdown 文档构建知识库，通过混合检索和大模型生成，为用户提供菜谱推荐、食材查询、制作步骤问答和多轮对话能力。

1. 项目总览

1.1 项目解决的问题

这个项目不是一个普通聊天机器人，而是一个面向食谱领域的 RAG 应用。它要解决的问题是：用户用自然语言询问菜谱、食材、制作步骤或菜品推荐时，系统能够先从本地食谱知识库中检索相关内容，再把检索到的内容交给大语言模型生成回答。

项目知识库来源于 `data/C8/cook` 下的 Markdown 食谱文档，当前约有 323 个 `.md` 文件。文档包含菜品名称、原料、步骤、难度、附加说明等内容，非常适合作为 RAG 的领域知识源。

1.2 核心能力

系统实现了以下功能：

1. 食谱文档加载与元数据抽取。
2. Markdown 结构感知切分。
3. 父子文档映射。
4. 向量索引构建与持久化。
5. 本地知识库缓存与启动加速。
6. 文件变化检测与更新触发。
7. 向量检索和 BM25 关键词检索。
8. RRF 融合重排。
9. 基于分类和难度的元数据过滤。
10. 查询路由，将问题分为列表、详情、通用三类。
11. 多轮对话历史缓存。
12. 上下文查询改写，解决“这个怎么做”等省略主语问题。
13. DeepSeek 大模型回答生成。

- 14. 非流式和流式回答输出。
- 15. Chainlit Web 聊天界面。
- 16. UI 中展示分析、检索、生成等处理步骤。

1.3 技术栈

层级	技术	作用
应用框架	Python	后端主语言
RAG 编排	LangChain	Document、Prompt、Chain、Retriever 等组件
文档切分	MarkdownHeaderTextSplitter	按 Markdown 标题结构切分
Embedding	BAAI/bge-small-zh-v1.5	中文语义向量化
向量库	FAISS	本地向量索引与相似度检索
关键词检索	BM25Retriever	关键词召回
重排	RRF	融合向量检索与 BM25 检索
LLM	DeepSeek Chat	回答生成、查询路由、查询改写
Web UI	Chainlit	聊天界面、流式输出、步骤展示
配置	dataclass + dotenv	集中管理参数与 API Key

1.4 一句话面试介绍

可以这样介绍：

“这个项目是一个面向食谱场景的 RAG 问答系统。我把本地 Markdown 食谱解析成 LangChain Document，按 Markdown 标题做结构化切分，然后用 bge-small-zh-v1.5 生成向量并存入 FAISS。同时为了弥补纯向量检索对菜名、食材关键词匹配不稳定的问题，我加入了 BM25，并用 RRF 做结果融合。查询进入系统后会先经过 LLM 路由和上下文改写，再根据分类、难度等元数据做过滤，最后把召回到的父文档拼成上下文交给 DeepSeek 生成回答。前端用 Chainlit 实现聊天界面、流式输出和检索过程展示。”

2. 代码结构

项目主要文件如下：

```
code/C8
  config.py
  main.py
  web_ui_recipe.py
  chainlit.md
  requirements.txt
  rag_modules
    __init__.py
    data_preparation.py
    index_construction.py
    retrieval_optimization.py
    generation_integration.py
  vector_index
```

各文件职责：

文件	主要职责
config.py	定义 RAGConfig，集中管理路径、模型、检索和生成参数
main.py	系统主入口，串联数据、索引、检索、生成模块
web_ui_recipe.py	Chainlit Web UI 入口，处理用户会话和消息
data_preparation.py	加载 Markdown，增强元数据，结构化切分，缓存文档状态
index_construction.py	初始化 Embedding，构建、保存、加载 FAISS 向量索引
retrieval_optimization.py	建立向量检索器和 BM25 检索器，执行混合检索和 RRF 重排
generation_integration.py	初始化 DeepSeek，完成查询路由、查询改写、Prompt 生成和流式输出

3. 整体架构与调用链路

3.1 模块分层

系统按职责拆成五个核心模块：

- 1. 配置层： RAGConfig

2. 数据准备层: `DataPreparationModule`
3. 索引构建层: `IndexConstructionModule`
4. 检索优化层: `RetrievalOptimizationModule`
5. 生成集成层: `GenerationIntegrationModule`

外层由 `RecipeRAGSystem` 统一编排。Web 界面由 `web_ui_recipe.py` 调用 `RecipeRAGSystem`。

3.2 启动流程

启动时调用链路如下：

```
RecipeRAGSystem()  
-> 检查 data_path 是否存在  
-> 检查 DEEPSEEK_API_KEY  
-> initialize_system()  
    -> 创建 DataPreparationModule  
    -> 创建 IndexConstructionModule  
        -> 初始化 HuggingFaceEmbeddings  
    -> 创建 GenerationIntegrationModule  
        -> 初始化 DeepSeek ChatOpenAI  
-> build_knowledge_base()  
    -> 尝试加载 FAISS 索引  
    -> 尝试加载文档缓存  
    -> 检查文件是否新增或修改  
    -> 命中缓存则复用  
    -> 否则加载文档、切分、建索引、保存缓存  
    -> 初始化 RetrievalOptimizationModule
```

3.3 一次问答流程

用户提问后，`ask_question()` 中的主链路如下：

用户问题

- > query_router 判断问题类型
- > query_rewrite 根据历史对话重写查询
- > _extract_filters_from_query 提取分类/难度过滤条件
- > hybrid_search 或 metadata_filtered_search
- > get_parent_documents 由子块找到完整食谱
- > 根据 route_type 选择回答方式
 - list: 直接返回菜名列表
 - detail: 生成分步骤回答
 - general: 生成基础回答
- > 返回普通字符串或流式生成器

面试时可以强调：这个链路把“检索准确性”和“生成质量”拆开处理。检索阶段尽量找到正确菜谱，生成阶段只基于召回内容组织答案，从而降低模型幻觉。

4. 配置管理实现

配置文件是 `config.py`。

`RAGConfig` 是一个 dataclass，包含：

```
data_path: str = "../..data/C8/cook"
index_save_path: str = "./vector_index"
embedding_model: str = "BAAI/bge-small-zh-v1.5"
llm_model: str = "deepseek-chat"
top_k: int = 3
temperature: float = 0.1
max_tokens: int = 2048
```

4.1 为什么这样设计

把路径、模型、检索数量和生成参数集中在配置类里，有几个好处：

1. 便于替换模型，比如从 `deepseek-chat` 换成其他 OpenAI 兼容模型。
2. 便于调参，比如修改 `top_k`、`temperature`、`max_tokens`。
3. 主流程代码不用写死参数，模块之间更解耦。
4. 后续可以很容易扩展为从 YAML、JSON 或环境变量加载配置。

4.2 API Key 校验

在 `RecipeRAGSystem.__init__()` 中，系统会检查：

```
if not os.getenv("DEEPSEEK_API_KEY"):
    raise ValueError("请设置 DEEPSEEK_API_KEY 环境变量")
```

这能在系统初始化阶段尽早暴露配置问题，避免用户真正提问时才发现模型无法调用。

面试回答要点：

“我把模型密钥放在环境变量里，不写入代码。启动阶段会做显式校验，这是一种基本的工程防御。后续如果生产化，可以把密钥接入 Secret Manager 或部署平台环境变量。”

5. 数据准备模块

对应代码：`rag_modules/data_preparation.py`

数据准备模块负责把原始 Markdown 食谱转成可检索的 LangChain `Document`，并补充元数据。

5.1 文档加载

`load_documents()` 会递归读取 `data_path` 下所有 `.md` 文件：

```
for md_file in data_path_obj.rglob("*.md"):
    with open(md_file, "r", encoding="utf-8") as f:
        content = f.read()
```

每个 Markdown 文件会被包装成一个 LangChain `Document`：

```
Document(
    page_content=content,
    metadata={
        "source": str(md_file),
        "parent_id": parent_id,
        "doc_type": "parent"
    }
)
```

这里的完整 Markdown 文件被称为父文档。父文档代表一道完整菜谱。

5.2 parent_id 的生成

代码用文档相对路径生成稳定 ID：

```
relative_path = Path(md_file).resolve().relative_to(data_root).as_posix()
parent_id = hashlib.md5(relative_path.encode("utf-8")).hexdigest()
```

这样做的意义是：

1. 同一个文件每次加载得到相同的 `parent_id`。
2. 能把切分后的多个子块关联回完整食谱。
3. 后续做缓存、去重、更新时有稳定标识。

面试时可以说：

“我没有用随机 UUID 作为父文档 ID，而是用相对路径做 MD5。这样父文档 ID 是确定性的，重启后仍然一致，方便父子文档映射和缓存恢复。”

5.3 元数据增强

`_enhance_metadata()` 会给每个文档补充三个关键字段：

1. `category`：菜品分类。
2. `dish_name`：菜品名称。
3. `difficulty`：难度等级。

分类来自路径目录，例如：

路径目录	中文分类
meat_dish	荤菜
vegetable_dish	素菜
soup	汤品
dessert	甜品
breakfast	早餐
staple	主食
aquatic	水产
condiment	调料
drink	饮品

菜品名称来自文件名：

```
doc.metadata["dish_name"] = file_path.stem
```

难度来自文档中的星级符号：

```
if "★★★★★" in content:
    difficulty = "非常困难"
elif "★★★★" in content:
    difficulty = "困难"
elif "★★★" in content:
    difficulty = "中等"
elif "★★" in content:
    difficulty = "简单"
elif "★" in content:
    difficulty = "非常简单"
else:
    difficulty = "未知"
```


5.4 为什么元数据重要

元数据不是给 LLM 看的装饰信息，而是检索阶段的重要控制信号。

例如用户问：

“推荐几个简单的素菜”

系统可以从问题中抽取：

```
{"category": "素菜", "difficulty": "简单"}
```

然后只在满足条件的候选文档中选结果。这样比只靠向量相似度更精确。

面试回答要点：

“RAG 不只是把文本丢进向量库。结构化元数据可以把自然语言查询转成过滤条件，尤其适合分类、难度、地区、时间、标签这类字段。它能减少无关召回，提高答案可控性。”

6. Markdown 结构感知分块

对应函数：`chunk_documents()` 和 `_markdown_header_split()`

6.1 为什么不直接固定长度切分

食谱文档通常天然有结构，例如：

```
# 宫保鸡丁
## 必备原料和工具
## 计算
## 操作
## 附加内容
```

如果按固定 token 或固定字符长度切分，可能会把“原料”和“步骤”切在一起，也可能把一个步骤截断。这样检索到的片段上下文不完整。

所以系统使用 LangChain 的 `MarkdownHeaderTextSplitter`，按 Markdown 标题结构切分。

6.2 切分配置

代码定义了三个标题层级：

```
headers_to_split_on = [
    ("#", "主标题"),
    ("##", "二级标题"),
    ("###", "三级标题")
]
```

并设置：

```
strip_headers=False
```

这表示切分后保留标题。保留标题有助于模型理解这个片段属于“原料”“操作”还是“附加内容”。

6.3 子块元数据

每个 chunk 都会带上：

字段	含义
chunk_id	子块唯一 ID
parent_id	对应父文档 ID
doc_type	标记为 child
chunk_index	在父文档中的位置
batch_index	在当前批次中的位置
chunk_size	子块长度

6.4 父子文档设计

系统检索时检索的是子块，但生成回答时使用完整父文档。

原因是：

- 1. 子块更短，适合向量检索，召回更精准。
- 2. 完整父文档信息更全，适合生成完整做法。
- 3. 检索和生成使用不同粒度，可以兼顾精度和上下文完整性。

get_parent_documents() 会根据检索到的子块找到父文档，并按父文档被命中的次数排序。

面试回答话术：

“我采用了 parent-child retrieval 思路。检索阶段用 chunk 提高召回精度，生成阶段回到 parent 文档，避免模型只看到一个局部片段导致回答不完整。比如用户问一道菜怎么做，检索可能命中操作片段，但回答还需要原料、工具和注意事项，所以要回到完整食谱。”

7. 缓存、持久化与更新机制

7.1 文档状态缓存

数据准备模块会把解析后的状态保存到：

```
data/C8/knowledge_cache.pkl
```

缓存内容包括：

```
{
    "documents": self.documents,
    "chunks": self.chunks,
    "parent_child_map": self.parent_child_map,
    "file_hashes": self.file_hashes
}
```

这能避免每次启动都重新读取和切分 Markdown。

7.2 向量索引持久化

索引模块会把 FAISS 保存到：

```
code/C8/vector_index
```

使用的是：

```
self.vectorstore.save_local(self.index_save_path)
```

加载时使用：

```
FAISS.load_local(  
    self.index_save_path,  
    self.embeddings,  
    allow_dangerous_deserialization=True  
)
```

7.3 启动时的三种情况

`build_knowledge_base()` 会同时尝试加载 FAISS 索引和文档缓存。

第一种情况：索引存在，缓存存在，文件无变化。

系统直接复用缓存和索引，实现快速启动。

第二种情况：索引存在，缓存存在，但发现文件新增或修改。

系统重新加载文档、重新切分、重新构建索引，并保存缓存。

第三种情况：索引或缓存缺失。

系统走全量构建流程：加载文档 -> 切分 -> 构建索引 -> 保存索引 -> 保存缓存。

7.4 文件变化检测

`check_for_updates()` 会遍历所有 Markdown 文件，并记录文件最后修改时间：

```
current_mtime = str(os.path.getmtime(md_file))
```

如果文件路径不存在于 `file_hashes` 中，或者修改时间变了，就认为文件有更新。

7.5 面试中的诚实说明

需要注意：当前代码的“增量更新”更准确地说是“变更检测后触发重建”。它能检测哪些文件发生变化，但发生变化后当前实现会重新加载所有文档并重建完整索引，而不是只对变更文件做真正的 add/update/delete。

面试时可以这样说：

“这个版本已经实现了文件变化检测和缓存复用。严格来说，当前增量更新还不是完全的局部更新，因为 FAISS 对删除和更新的管理比较麻烦，所以发现变化后先重建索引，保证正确性。下一步我会把文档 ID 和向量 ID 建立映射，对新增文档调用 `add_documents`，对修改文档先删除旧向量再插入新向量，或者切换到更适合增删改的向量数据库，比如 Milvus、Qdrant 或 Chroma。”

8. 向量索引构建

对应代码: `rag_modules/index_construction.py`

8.1 Embedding 模型

系统使用:

```
BAAI/bge-small-zh-v1.5
```

这是一个中文语义表示模型, 适合中文问题和中文文档的相似度检索。

初始化方式:

```
HuggingFaceEmbeddings(  
    model_name=self.model_name,  
    model_kwargs={"device": "cpu"},  
    encode_kwargs={"normalize_embeddings": True}  
)
```

8.2 为什么 normalize_embeddings

`normalize_embeddings=True` 会把向量归一化。归一化后, 向量相似度计算更接近余弦相似度, 能减少向量长度差异带来的影响。

面试回答:

“对于语义检索, 通常更关注方向相似度而不是向量长度。归一化可以让相似度计算更稳定, 尤其是不同长度文本的 embedding 之间进行比较时。”

8.3 FAISS 索引构建

构建索引的核心代码:

```
self.vectorstore = FAISS.from_documents(  
    documents=chunks,  
    embedding=self.embeddings  
)
```

LangChain 会对每个 chunk 调用 embedding 模型, 得到向量后写入 FAISS。

8.4 保存与加载

保存：

```
self.vectorstore.save_local(self.index_save_path)
```

加载：

```
self.vectorstore = FAISS.load_local(...)
```

这样系统不需要每次启动都重新 embedding 323 篇文档。

8.5 为什么选择 FAISS

FAISS 的优点：

1. 本地部署简单，不需要额外服务。
2. 对中小规模知识库检索速度快。
3. 和 LangChain 集成方便。
4. 适合教学项目、原型验证、本地 Demo。

局限：

1. 多用户生产部署能力不如专用向量数据库。
2. 元数据过滤能力较弱，需要应用层处理。
3. 增删改和索引版本管理没有 Milvus、Qdrant 等数据库完善。

9. 混合检索与 RRF 重排

对应代码：`rag_modules/retrieval_optimization.py`

9.1 为什么需要混合检索

纯向量检索适合理解语义，比如：

“这个菜怎么做”

“有没有容易上手的晚餐”

但它对具体词匹配不一定稳定。例如菜名、食材名、调料名可能需要精确匹配：

“鱼香肉丝”

“不放酱油”

“土豆”

BM25 正好擅长关键词匹配。因此系统把向量检索和 BM25 结合起来。

9.2 两路召回

向量检索器：

```
self.vector_retriever = self.vectorstore.as_retriever(  
    search_type="similarity",  
    search_kwargs={"k": 5}  
)
```

BM25 检索器：

```
self.bm25_retriever = BM25Retriever.from_documents(  
    self.chunks,  
    k=5  
)
```

查询时分别调用：

```
vector_docs = self.vector_retriever.invoke(query)  
bm25_docs = self.bm25_retriever.invoke(query)
```

9.3 RRF 重排

系统使用 RRF，也就是 Reciprocal Rank Fusion，把两路结果融合。

分数公式：

$$\text{score} = 1 / (k + \text{rank} + 1)$$

如果同一个文档同时被向量检索和 BM25 检索命中，它的分数会累加，因此排序会更靠前。

代码中默认 `k=60`，用于平滑排名差异。

9.4 RRF 的优点

1. 不要求不同检索器分数在同一个尺度上。
2. 只依赖排名，不依赖原始相似度分数。
3. 实现简单，效果稳定。
4. 特别适合把语义召回和关键词召回融合。

面试回答话术：

“向量检索和 BM25 的分数不可直接比较，一个是语义相似度，一个是词频相关性。所以我没有直接加权原始分数，而是用 RRF 基于排名融合。这样只要文档在多个召回器里排名都靠前，就会获得更高最终分数。”

10. 元数据过滤检索

10.1 过滤条件抽取

在 `main.py` 的 `_extract_filters_from_query()` 中，系统会检查用户问题是否包含分类或难度关键词。

分类关键词来自：

```
DataPreparationModule.get_supported_categories()
```

难度关键词来自：

```
DataPreparationModule.get_supported_difficulties()
```

如果用户问题里出现“素菜”“简单”等词，就生成过滤条件：

```
{"category": "素菜", "difficulty": "简单"}
```

10.2 过滤执行方式

`metadata_filtered_search()` 先扩大召回：

```
docs = self.hybrid_search(query, top_k * 3)
```


然后在候选结果中逐个检查 metadata 是否匹配。

这样做的原因是：如果一开始只召回 `top_k=3`，可能过滤完就没有结果。先召回更多候选，再过滤，更容易得到满足条件的文档。

10.3 典型例子

用户问：

“推荐几个简单的素菜”

系统流程：

- 1. 路由判断为 `list`。
- 2. 查询不做改写。
- 3. 抽取过滤条件：`category=素菜`，`difficulty=简单`。
- 4. 混合检索召回更多候选。
- 5. 只保留元数据匹配的文档。
- 6. 找到父文档。
- 7. 返回菜名列表。

面试回答要点：

“元数据过滤让检索从纯相似度排序变成了条件约束下的排序。对于推荐类需求，这比完全依赖大模型判断更可靠，也更可解释。”

11. 查询路由

对应代码：`GenerationIntegrationModule.query_router()`

11.1 路由类型

系统将问题分为三类：

类型	含义	示例
<code>list</code>	用户想要菜品列表或推荐	推荐几个素菜、有什么川菜
<code>detail</code>	用户想要具体制作方法	宫保鸡丁怎么做、需要什么食材
<code>general</code>	一般知识或技巧问题	什么是川菜、如何炒菜不粘锅

11.2 实现方式

系统通过 Prompt 让 LLM 分类：

根据用户的问题，将其分类为 `list`、`detail` 或 `general`。
请只返回分类结果。

如果模型返回的不是合法类别，代码默认回退到 `general`：

```
if result in ["list", "detail", "general"]:  
    return result  
else:  
    return "general"
```

11.3 路由结果如何影响后续流程

不同路由走不同回答策略：

1. `list`：直接从父文档中提取菜品名称，返回列表，不调用 LLM 生成文。
2. `detail`：调用分步骤 Prompt，生成菜品介绍、食材、步骤、技巧。
3. `general`：调用基础问答 Prompt，生成普通解释类回答。

11.4 面试可讲点

“查询路由能避免所有问题都走同一个 Prompt。比如推荐菜名时没必要让大模型长篇生成，直接返回检索到的菜名更快、更稳定；制作步骤问题则需要更结构化的 Prompt。路由让系统在成本、速度和回答质量之间做动态选择。”

12. 多轮对话与查询改写

对应代码：`GenerationIntegrationModule.query_rewrite()`

12.1 为什么需要查询改写

多轮对话中，用户经常省略主语。例如：

用户：红烧肉是什么菜？
助手：红烧肉是一道经典家常菜...
用户：那具体怎么做？

第二个问题“那具体怎么做”本身没有菜名。如果直接检索，很可能召回错误菜谱。

12.2 历史记录窗口

系统会传入 `chat_history`，并只取最近 6 条消息，也就是最近 3 轮对话：

```
recent_history = chat_history[-6:]
```

这样既能保留上下文，又能避免 Prompt 过长。

12.3 改写 Prompt 的目标

Prompt 要求模型：

1. 如果问题包含“它”“这个”等代词，要参考历史记录补全菜品名。
2. 如果问题过于宽泛，可以重写成更适合检索的查询。
3. 如果问题已经明确，就返回原查询。

例如：

“具体怎么做” -> “红烧肉具体怎么做”
“有饮品推荐吗” -> “简单饮品制作方法”
“宫保鸡丁怎么做” -> “宫保鸡丁怎么做”

12.4 面试回答话术

“RAG 的检索质量很依赖 query。如果用户的 query 缺少实体，比如‘这个怎么做’，向量检索会失去目标。所以我在检索前加了 query rewrite，把用户当前问题和短期历史一起交给 LLM，让它补全省略的主语。这一步属于检索前优化，可以显著提升多轮问答的召回准确率。”

13. 父文档召回与去重排序

对应代码：`DataPreparationModule.get_parent_documents()`

13.1 实现逻辑

检索模块返回的是子块列表。生成回答前，系统会根据每个子块的 `parent_id` 找到对应完整食谱。

核心逻辑：

1. 遍历检索到的 child chunks。
2. 读取每个 chunk 的 `parent_id`。
3. 统计每个 parent 被命中的次数。
4. 根据 `parent_id` 在 `self.documents` 中找到父文档。
5. 按命中次数排序。
6. 返回去重后的父文档列表。

13.2 为什么按命中次数排序

如果一道菜的多个章节都被检索命中，说明这道菜和用户问题更相关。因此父文档命中次数可以作为一个简单的相关性信号。

例如用户问：

“鱼香肉丝需要什么食材，具体怎么做？”

如果同一道菜的“原料”和“操作”两个 chunk 都被命中，它比只命中一个片段的文档更值得排在前面。

13.3 面试可讲点

“父子映射不仅用于找回完整上下文，也用于去重。否则多个 chunk 来自同一道菜，直接把它们全部交给 LLM 会造成重复上下文。通过 `parent_id` 去重后，Prompt 更紧凑，信息也更完整。”

14. 回答生成模块

对应代码： `rag_modules/generation_integration.py`

14.1 LLM 初始化

系统用 `langchain_openai.ChatOpenAI` 调 DeepSeek：

```
self.llm = ChatOpenAI(  
    model=self.model_name,  
    temperature=self.temperature,  
    max_tokens=self.max_tokens,  
    api_key=api_key,  
    base_url="https://api.deepseek.com",  
)
```

虽然类名是 `ChatOpenAI`，但由于 DeepSeek 提供 OpenAI 兼容接口，所以可以通过 `base_url` 接入。

14.2 基础回答

`generate_basic_answer()` 用于通用问题。Prompt 要求模型：

1. 扮演专业烹饪助手。
2. 根据食谱信息回答。
3. 如果信息不足，要诚实说明。

这类回答适合：

鱼香肉丝需要什么材料？
什么是川菜？
糖醋汁怎么调？

14.3 分步骤回答

`generate_step_by_step_answer()` 用于具体做法问题。Prompt 引导模型输出：

1. 菜品介绍。
2. 所需食材。
3. 制作步骤。
4. 制作技巧。

同时要求：

1. 根据实际内容灵活调整结构。
2. 不要强行填充无关内容。
3. 不要重复制作步骤里的信息。
4. 如果没有额外技巧，可以省略技巧部分。

14.4 列表回答

`generate_list_answer()` 不调用 LLM，而是直接从 `context_docs` 的 metadata 中提取 `dish_name`。

这样做有几个优点：

1. 速度快。
2. 成本低。
3. 不容易编造菜名。
4. 推荐结果都来自知识库。

14.5 上下文构造

`_build_context()` 会把父文档转成 Prompt 中的上下文，格式类似：

```
【食谱 1】 宫保鸡丁 | 分类：荤菜 | 难度：中等  
文档内容...
```

并限制总长度：

```
max_length = 2000
```

如果超过长度就停止追加，避免 Prompt 太长。

14.6 面试可讲点

“我在生成阶段把元数据也放进上下文，例如菜名、分类、难度。这样模型不仅看到正文，还知道这份文档在领域里的结构属性。对于回答推荐理由、难度判断、分类问题很有帮助。”

15. 流式输出

15.1 后端流式生成

生成模块提供两个流式函数：

```
generate_basic_answer_stream()  
generate_step_by_step_answer_stream()
```

它们使用 LangChain 的：

```
chain.stream(...)
```

每次产生一个文本片段：

```
for chunk in chain.stream(...):  
    yield chunk
```

15.2 Chainlit 前端流式展示

在 `web_ui_recipe.py` 中，系统先创建空消息：

```
response_msg = cl.Message(content="")  
await response_msg.send()
```

然后逐块写入：

```
for chunk in generator:  
    full_response += chunk  
    await response_msg.stream_token(chunk)
```

这会形成打字机效果，用户不用等完整回答生成完才看到内容。

15.3 面试可讲点

“流式输出本身不提升模型质量，但能显著提升用户体验。特别是 RAG 回答通常较长，用户看到系统持续输出，会感知到系统在工作，而不是卡住。”

16. Chainlit Web UI

对应代码： `web_ui_recipe.py`

16.1 全局知识库单例

代码定义：

```
global_rag_system = None
```

第一次用户打开页面时才初始化：

```
if global_rag_system is None:
    global_rag_system = await asyncio.to_thread(init_system)
```

后续用户或刷新页面复用同一个知识库实例。

16.2 为什么用全局单例

RAG 知识库初始化比较耗时，包括加载 embedding 模型、加载 FAISS、构建 BM25 等。如果每个用户会话都重新初始化，性能很差。

全局单例的好处：

1. 避免重复加载模型和索引。
2. 多个用户共享同一份知识库。
3. 提升页面刷新后的响应速度。

需要注意：如果生产环境有多进程部署，每个进程仍会有自己的内存实例。更进一步可以把向量库和缓存放到独立服务。

16.3 会话级历史记录

虽然知识库是全局共享的，但聊天历史是每个用户独立的：

```
cl.user_session.set("chat_history", [])
```

每次回答后追加：

```
chat_history.append(("human", message.content))
chat_history.append(("ai", full_response))
```

并保留最近 10 条：

```
if len(chat_history) > 10:
    chat_history = chat_history[-10:]
```


这避免不同用户之间串话。

16.4 Step 展示

Chainlit 中使用 `cl.Step` 展示处理过程：

1. 正在分析您的需求。
2. 正在翻阅菜谱库。
3. 正在为您生成回答。

每个 Step 会输出当前阶段状态，例如识别到的过滤条件、找到的参考菜谱名称等。

这属于可解释性和用户体验优化。用户能看到系统不是直接“凭空回答”，而是经历了分析和检索。

16.5 异步与同步桥接

Chainlit 是异步事件模型，但 RAG 初始化和 LLM 调用多数是同步操作。代码使用：

```
await asyncio.to_thread(...)
```

把耗时同步任务放到线程中执行，避免阻塞 UI。

面试回答话术：

“前端框架是 async 的，但底层 LangChain 和模型调用很多是同步的。如果直接在 async handler 里执行，会阻塞事件循环，导致页面状态不刷新。所以我用 `asyncio.to_thread` 把初始化、路由、检索等同步任务放到后台线程里执行。”

17. 命令行交互模式

除了 Chainlit Web UI，`main.py` 里还保留了命令行交互：

```
run_interactive()
```

它会：

1. 初始化系统。
2. 构建知识库。
3. 循环读取用户输入。
4. 让用户选择是否流式输出。

5. 调用 `ask_question()` 返回答案。

这个模式适合本地调试，不依赖 Web 页面。

面试时可以说：

“我保留了 CLI 模式作为最小可运行入口，方便排查 RAG 主链路。Web UI 出问题时，可以先用 CLI 判断是界面问题还是后端 RAG 逻辑问题。”

18. 辅助能力

18.1 按分类搜索菜品

`search_by_category(category, query="")` 会使用：

```
filters = {"category": category}
metadata_filtered_search(...)
```

然后从结果中提取去重菜名。

这个功能适合做快捷入口，例如“早餐”“素菜”“汤品”按钮。

18.2 查询指定菜品食材

`get_ingredients_list(dish_name)` 会先检索菜名，再调用基础回答生成：

```
generate_basic_answer(f"{dish_name}需要什么食材?", docs)
```

这是一个面向具体任务的封装函数。

18.3 统计信息

`get_statistics()` 会统计：

1. 文档总数。
2. 文本块总数。
3. 分类分布。
4. 难度分布。
5. 平均 chunk 长度。

这类信息可以帮助评估知识库规模，也方便调试切分效果。

18.4 元数据导出

`export_metadata(output_path)` 可以把文档元数据导出为 JSON，包括来源路径、菜名、分类、难度、正文长度。

这适合后续做数据质量检查，例如检查哪些文档难度未知、哪些分类不在映射中。

19. 面试高频问题与回答

19.1 你这个 RAG 系统的完整流程是什么？

回答：

“用户问题进入系统后，先由 LLM 做查询路由，判断是推荐列表、具体做法还是一般问题。对于非列表问题，会结合最近几轮对话做 query rewrite，补全省略主语。然后系统从问题中抽取分类、难度等元数据过滤条件。如果有过滤条件，就执行带过滤的混合检索；否则执行普通混合检索。混合检索包括 FAISS 向量检索和 BM25 关键词检索，之后用 RRF 融合重排。检索返回的是 chunk，系统再通过 parent_id 找回完整食谱文档，最后根据路由类型选择列表回答、基础回答或分步骤回答，并通过 Chainlit 流式展示给用户。”

19.2 为什么要用混合检索？

回答：

“向量检索擅长语义相似，但对具体菜名、食材名、调料名这类关键词不一定稳定。BM25 擅长词面匹配，但不理解语义。食谱场景既有语义需求，比如‘简单家常菜’，也有精确词需求，比如‘鱼香肉丝’或‘土豆’。所以我把 FAISS 向量检索和 BM25 结合，再用 RRF 做排名融合。”

19.3 为什么用 RRF，而不是直接加权两个分数？

回答：

“向量相似度和 BM25 分数不是一个尺度，直接加权会有归一化问题。RRF 只依赖排名，不依赖原始分数。一个文档如果在两路检索中排名都靠前，就会得到更高融合分数。这种方式实现简单，对多检索器融合比较稳定。”

19.4 为什么要做父子文档映射？

回答：

“因为检索和生成适合不同粒度。检索时 chunk 越聚焦越容易匹配问题，但生成时只给 chunk 可能信息不完整。比如检索命中‘操作’章节，但回答做法还需要原料和工具。所以我检索子块，再根据 parent_id 找回完整食谱，让生成阶段有完整上下文。”

19.5 如何减少幻觉？

回答：

“主要从三方面控制。第一，回答 Prompt 明确要求根据食谱信息回答，信息不足要诚实说明。第二，列表推荐不让 LLM 编造，而是直接从检索文档的 dish_name 元数据里提取。第三，使用混合检索、元数据过滤和父文档召回提高上下文相关性。RAG 系统里，减少幻觉的关键不是只靠 Prompt，而是先保证检索上下文正确。”

19.6 如何处理多轮对话？

回答：

“Chainlit 的 user_session 里维护每个用户自己的 chat_history。每次请求会把最近几轮历史传给 query_rewrite。如果用户问‘这个怎么做’这类省略主语的问题，模型会参考历史记录补全菜品名，再拿重写后的查询去检索。回答生成时，流式 Prompt 里也带了 MessagesPlaceholder，可以把历史消息传给模型。”

19.7 如何做知识库持久化？

回答：

“持久化分两部分。文档解析状态，包括 documents、chunks、parent_child_map、file_hashes，会用 pickle 保存到 knowledge_cache.pkl。向量索引用 FAISS 的 save_local 保存到 vector_index。启动时先尝试加载索引和缓存，如果都存在且文件无变化，就直接复用，实现快速启动。”

19.8 当前增量更新做到了什么程度？

回答：

“当前已经能检测哪些 Markdown 文件新增或修改，基于文件修改时间做指纹。但发现变化后，为了保证正确性，目前会重新加载文档并重建索引。严格说这是变更检测加重建，不是完全局部增量。后续可以把 chunk_id 和向量库 ID 对齐，支持新增、删除、修改的局部更新，或者换成更支持增删改的向量数据库。”

19.9 如果检索不准，你会怎么排查？

回答：

“我会按链路逐步排查。第一看 query router 是否分类错误。第二看 query rewrite 是否把问题改坏。第三看元数据过滤是否过严导致召回为空。第四分别查看向量检索和 BM25 的 top results，看是语义召回问题还是关键词召回问题。第五检查 chunk 切分是否过碎或缺少标题。第六评估 embedding 模型是否适合当前中文领域。如果还不够，可以加 reranker 或调整 top_k。”

19.10 如果数据规模扩大 10 倍，怎么优化？

回答：

“首先会把 FAISS 本地索引迁移到专用向量数据库，例如 Milvus、Qdrant 或 Elasticsearch/OpenSearch 向量检索，支持分片、持久化、过滤和增删改。第二会做真正增量更新，避免重建全量索引。第三会增加 reranker，把初召回 top_k 提大，再用交叉编码器重排。第四会做更好的中文分词和结构化字段抽取，例如食材、做法、菜系。第五会给查询路由、检索结果和生成结果加日志和评估集，持续评估 recall、precision 和 answer faithfulness。”

19.11 为什么选择 DeepSeek？

回答：

“DeepSeek 提供 OpenAI 兼容接口，接入成本低，可以直接用 LangChain 的 ChatOpenAI，通过 base_url 指向 DeepSeek API。它中文能力较好，成本也适合应用原型。代码里模型名来自配置，所以后续可以替换成其他兼容模型。”

19.12 这个项目和普通 ChatBot 的区别是什么？

回答：

“普通 ChatBot 主要依赖模型自身知识，而这个项目是知识库增强。模型回答前必须先检索本地食谱文档，答案基于文档上下文生成。这样能回答模型训练数据里未必包含的本地菜谱，也能把推荐结果限制在知识库范围内，减少编造。”

20. 当前代码的风险点与可优化方向

20.1 依赖清单不完整

`requirements.txt` 中包含 LangChain、FAISS、sentence-transformers、openai 等依赖，但当前 Web UI 还需要：

```
chainlit
python-dotenv
```

如果新环境安装依赖，可能会因为缺少这两个包而启动失败。

20.2 中文 BM25 分词问题

当前使用 `BM25Retriever.from_documents()`，没有显式中文分词。中文文本没有天然空格，BM25 可能无法发挥最佳效果。

优化方向：

1. 接入 jieba 分词。
2. 对菜名、食材名维护词典。
3. 使用支持中文分析器的 Elasticsearch/OpenSearch。

20.3 真正增量更新还未完成

当前检测到文件变化后会重建索引。后续可以：

1. 为每个 chunk 生成稳定 ID。
2. 维护 chunk_id 到向量库 ID 的映射。
3. 新增文档调用 add。
4. 修改文档先删除旧 chunk 再插入新 chunk。
5. 删除文档时同步删除对应向量。

20.4 `allow_dangerous_deserialization=True`

FAISS 加载时设置了：

```
allow_dangerous_deserialization=True
```

这在本地可信文件场景可以接受，但生产环境要谨慎，因为反序列化不可信文件存在安全风险。

面试时可以说明：

“这是 LangChain 加载本地 FAISS pickle 元数据的要求。生产环境要确保索引文件来源可信，或者改用更安全的持久化方案。”

20.5 category labels 使用 set 导致顺序不稳定

代码中：

```
CATEGORY_LABELS = list(set(CATEGORY_MAPPING.values()))
```

`set` 会导致顺序不稳定。虽然功能不一定受影响，但如果要做展示或稳定匹配，建议改成：

```
CATEGORY_LABELS = list(CATEGORY_MAPPING.values())
```

20.6 Chainlit 中有未使用函数

`web_ui_recipe.py` 中定义了：

```
def fetch_chunks():  
    return [chunk for chunk in generator]
```

但后面实际没有使用。可以删除，保持代码简洁。

20.7 流式生成仍可能阻塞事件循环

虽然部分同步操作用了 `asyncio.to_thread`，但生成器遍历这里：

```
for chunk in generator:  
    await response_msg.stream_token(chunk)
```

底层模型调用如果是同步阻塞，仍可能影响异步事件循环。后续可以改成异步流式 API，或者用队列把同步生成线程和异步 UI 消费解耦。

20.8 Prompt 上下文长度固定为字符数

`_build_context()` 用 `max_length=2000` 按字符数截断，而不是按 token 截断。生产环境更建议使用 tokenizer 估算 token 数，避免超出模型上下文窗口，也能更精确控制成本。

21. 可以扩展成更强项目的方向

21.1 引入 reranker

当前是向量 + BM25 + RRF。后续可以在融合召回后加入 reranker，例如 bge-reranker，流程变成：

```
向量/BM25 初召回 -> RRF 融合 -> reranker 精排 -> 父文档召回 -> 生成
```

reranker 能更准确判断 query 和候选文档的相关性。

21.2 使用结构化食材抽取

可以从 Markdown 中抽取：

1. 食材。
2. 调料。
3. 烹饪方式。
4. 菜系。
5. 口味。
6. 是否需要烤箱、空气炸锅等工具。

这些字段可以进入元数据或知识图谱，让用户能问：

家里有土豆和番茄，能做什么？
有没有不放辣椒的荤菜？
推荐几道需要炖的菜。

21.3 GraphRAG

可以构建图谱：

菜谱 -> 包含 -> 食材
菜谱 -> 属于 -> 分类
菜谱 -> 使用 -> 烹饪技法
菜谱 -> 具有 -> 难度

当用户问复杂约束问题时，先做实体识别和图谱查询，再结合向量检索。

21.4 多模态能力

可以让用户上传图片：

1. 上传菜品图片，识别可能的菜名。
2. 上传食材图片，识别食材组合。
3. 根据识别结果检索菜谱。
4. 生成制作建议。

21.5 评估体系

生产级 RAG 需要评估：

1. 检索召回率。
2. 检索准确率。
3. rerank 后 MRR 或 NDCG。
4. 回答忠实度。
5. 答案完整性。
6. 用户满意度。

可以构建一批标准问题，例如：

宫保鸡丁怎么做？
推荐几个简单素菜。
鱼香肉丝需要什么食材？
这个菜难不难？

然后人工标注期望召回文档，定期跑评估。

22. 面试时建议强调的亮点

1. 不是简单调用 LLM，而是完整 RAG 工程链路。
2. 使用 Markdown 结构化切分，符合食谱文档特点。
3. 使用父子文档映射，兼顾检索精度和生成完整性。
4. 使用 FAISS 做本地持久化，降低启动成本。
5. 使用 BM25 + 向量检索 + RRF，提升召回稳定性。
6. 支持元数据过滤，提高推荐类问题准确性。
7. 有查询路由，能根据问题类型选择不同回答策略。
8. 有多轮对话历史和 query rewrite，能处理上下文指代。
9. Web UI 展示分析、检索、生成过程，用户体验更好。
10. 能清楚说出当前不足和下一步优化，而不是只讲优点。

23. 快速背诵版

如果面试官让你 1 分钟介绍项目，可以说：

“我做的是一个食谱领域的 RAG 系统。数据来自本地 Markdown 菜谱，我先把每篇菜谱加载成 LangChain Document，并根据路径和内容抽取菜品分类、名称、难度等元数据。切分时没有用固定长度，而是用 MarkdownHeaderTextSplitter 按标题结构切分，并维护 parent-child 映射。索引用 bge-small-zh-v1.5 生成中文向量，存入 FAISS，并支持本地持久化和缓存加载。检索时结合 FAISS 语义检索和 BM25 关键词检索，用 RRF 融合结果，再根据分类和难度做元数据过滤。用户问题进入系统后，会先通过 LLM 做查询路由，必要时结合历史对话做 query rewrite，最后把召回到的完整父文档作为上下文交给 DeepSeek 生成回答。前端用 Chainlit 实现聊天、流式输出和检索步骤展示。”

如果面试官追问项目亮点，可以说：

“我觉得这个项目的重点不是单纯接了一个大模型，而是把 RAG 里的几个关键工程点串起来了：结构化切分、父子文档映射、混合检索、RRF 重排、元数据过滤、查询路由、上下文改写和流式交互。这些设计都是为了解决实际应用中的召回准确性、回答完整性和用户体验问题。”

24. 结论

这个 C8 项目已经具备一个完整 RAG 应用的主要组成部分：数据解析、切分、向量化、索引持久化、混合检索、重排、元数据过滤、查询改写、大模型生成和 Web 交互。它非常适合作为 AI 开发岗位面试项目，因为它覆盖了 LLM 应用开发中常见的核心问题：如何组织知识库、如何提高检索质量、如何减少幻觉、如何处理多轮对话、如何做系统工程化和用户体验优化。

面试准备时，不需要死记所有代码细节，但要能围绕一条主线讲清楚：

文档怎么进来 -> 怎么切 -> 怎么建索引 -> 怎么检索 -> 怎么过滤和重排 -> 怎么生成 -> 怎么展示
-> 有什么不足和优化方向

只要这条主线讲顺，面试官继续追问某个点时，就能自然展开到对应模块的实现细节。